

Victor Hugo Alves Vaz

Atividade – Árvore Binária de Busca

Aparecida de Goiânia - GO

2026

Victor Hugo Alves Vaz

Atividade – Árvore Binária de Busca

UNIFAN - Centro Educacional Alfredo Nasser

Instituto de Ciências Exatas Laboratório de Informática 2

Professor:Ricardo Vilaverde de Oliveira

Aparecida de Goiânia - GO

2026

Resumo

O projeto consiste no desenvolvimento de um programa para implementação de uma Árvore Binária de Busca (ABB). O sistema permite ao usuário inserir valores, realizar buscas e visualizar a estrutura hierárquica da árvore.

O programa também implementa diferentes formas de percorrer a árvore: pré-ordem, em ordem, pós-ordem, BFS (busca em largura) e DFS (busca em profundidade). Além disso, é possível calcular a altura da árvore e exibir sua estrutura, identificando os nós à esquerda e à direita.

As operações são disponibilizadas por meio de um menu interativo, que permanece em execução até que o usuário escolha a opção de sair. O código é organizado em métodos, facilitando a compreensão, manutenção e reutilização das funcionalidades.

Palavras-chave: Árvore Binária de Busca; Estrutura de Dados; Algoritmos; Busca; Inserção; Percursos em Árvores; BFS; DFS; Pré-ordem; Em ordem; Pós-ordem; Altura da Árvore.

Lista de ilustrações

Figura 1 – Insire o Valor	51
Figura 2 – Busca De Valor	53
Figura 3 – Pré _o rdem	55
Figura 4 – Valor em ordem	56
Figura 5 – Valor em Pós ordem	57
Figura 6 – BFS	58
Figura 7 – DFS	59
Figura 8 – Altura	60
Figura 9 – Estrutura Da Arvore	61
Figura 10 – Encerrando o programa	62
Figura 11 – Opção invalida	64
Figura 12 – Menu Do Terminal	80

Lista de tabelas

Sumário

0.1	Introdução	11
0.2	Objetivos	11
I	METODOLOGIA	13
1	CLASSE ÁRVORE BINARIA DE BUSCA	15
1.0.1	Representação da árvore	15
1.1	A classe ArvoreBinariaBusca	16
1.2	Construtor	16
1.3	Inserção	17
1.3.1	Posição vazia	17
1.3.2	Valor menor	18
1.3.3	Valor maior	18
1.3.4	Valor duplicado	18
1.4	Busca Recursiva	18
1.5	Busca Iterativa	20
1.6	Percursos da Árvore	21
1.7	Pré-ordem	21
1.8	Em ordem	22
1.9	Pós-ordem	23
1.10	BFS – Busca em Largura	23
1.11	DFS – Busca em Profundidade	24
1.12	Altura da Árvore	26
1.13	Visualização da Estrutura	27
1.14	Conclusão	27
1.15	Visualização Do Código	29
2	CLASSE NO	37
2.0.1	Declaração da classe	38
2.0.2	Atributo valor	39
2.0.3	Atributo esquerda	39
2.0.4	Atributo direita	40
2.1	Construtor da Classe	40
2.1.1	this.valor = valor	41
2.1.2	this.esquerda = null	41
2.1.3	this.direita = null	42

2.2	Representação de um Novo Nó	42
2.3	Conectando Nós	43
2.4	Relação com a Classe <code>ArvoreBinariaBusca</code>	43
2.5	Estrutura Completa	44
2.6	Resumo	45
3	CLASSE MAIN	47
3.1	Método <code>main</code>	47
3.2	Criação da Árvore	48
3.3	Criação do Scanner	48
3.4	Variável de Opção	49
3.5	Estrutura <code>do-while</code>	49
3.6	Verificação da Entrada	50
3.7	Estrutura <code>switch</code>	50
3.8	Opção 1 – Inserir Valor	51
3.9	Opção 2 – Buscar Valor	52
3.10	Opção 3 – Pré-ordem	54
3.11	Opção 4 – Em Ordem	55
3.12	Opção 5 – Pós-ordem	56
3.13	Opção 6 – BFS	57
3.14	Opção 7 – DFS	58
3.15	Opção 8 – Altura	59
3.16	Opção 9 – Estrutura da Árvore	61
3.17	Opção 10 – Encerrar	62
3.18	Opção Inválida	63
3.19	Fechamento do Scanner	63
3.20	Método <code>exibirMenu</code>	64
3.21	Fluxo Geral do Programa	66
3.22	Relação entre as Classes	66
3.23	Conclusão	67
II	RESULTADOS	71
4	RESULTADO DA EXECUÇÃO DO PROGRAMA	73
4.0.1	Menu Principal	73
4.0.2	Inserção de Valores	73
4.0.3	Busca de Valores	74
4.0.4	Percursos da Árvore	75
4.0.4.1	Pré-ordem	75

4.0.4.2	Em ordem	75
4.0.4.3	Pós-ordem	75
4.0.5	Busca em Largura – BFS	76
4.0.6	Busca em Profundidade – DFS	76
4.0.7	Cálculo da Altura	77
4.0.8	Visualização da Estrutura	78
4.0.9	Encerramento do Programa	78
4.1	Exemplo Completo de Execução	79
4.2	Conclusão da Execução	79
4.3	Conclusão	80

0.1 Introdução

As árvores binárias de busca são estruturas de dados fundamentais no estudo de algoritmos e estruturas computacionais, sendo utilizadas para organizar e armazenar valores de maneira hierárquica. Sua principal característica consiste em manter os elementos menores à esquerda e os maiores à direita de cada nó, facilitando operações de inserção e busca.

Neste trabalho, é desenvolvido um programa capaz de criar e manipular uma Árvore Binária de Busca, permitindo ao usuário inserir e pesquisar valores, visualizar a estrutura da árvore e calcular sua altura. O sistema também disponibiliza diferentes formas de percorrer os nós, incluindo os percursos em pré-ordem, em ordem, pós-ordem, busca em largura (BFS) e busca em profundidade (DFS).

O programa é organizado em métodos específicos para cada operação, proporcionando uma estrutura mais clara e facilitando a compreensão e manutenção do código. Dessa forma, o projeto busca aplicar na prática os conceitos de árvores binárias de busca e seus principais algoritmos de manipulação e percurso.

0.2 Objetivos

O objetivo deste trabalho é desenvolver um programa que implemente uma Árvore Binária de Busca, aplicando os principais conceitos relacionados a essa estrutura de dados. O sistema deverá permitir a inserção e busca de valores, a visualização da estrutura hierárquica da árvore, o cálculo de sua altura e a execução dos principais percursos, como pré-ordem, em ordem, pós-ordem, BFS e DFS.

Além disso, busca-se organizar o programa em métodos independentes, facilitando a compreensão do código e demonstrando, na prática, o funcionamento dos algoritmos de inserção, busca e percurso em uma árvore binária de busca.

Parte I

Metodologia

1 Classe Árvore Binária De Busca

A classe apresentada implementa uma **Árvore Binária de Busca**, também conhecida como *Binary Search Tree (BST)*.

Uma árvore binária é uma estrutura de dados na qual cada nó pode possuir, no máximo, dois filhos:

- filho esquerdo;
- filho direito.

Na Árvore Binária de Busca existe uma regra adicional:

valores menores → sub a rvore esquerda valores maiores → sub a rvore direita

Valores iguais são ignorados pela implementação apresentada.

1.0.1 Representação da árvore

Considere a inserção dos valores:

50 ,
30 ,
70 ,
20 ,
40 ,
60 ,
80

A árvore resultante será:

```

50
/
30 70
/ \ /
20 40 60 80

```

O elemento 50 é a raiz da árvore. Os valores menores que 50 estão à esquerda, enquanto os valores maiores estão à direita.

1.1 A classe ArvoreBinariaBusca

A classe possui uma variável chamada raiz:

```
private No raiz;
```

Essa variável representa o primeiro nó da árvore.

O tipo No deve representar cada elemento da árvore. Uma possível implementação seria:

```
public class No {  
    int valor;  
    No esquerda;  
    No direita;  
  
    public No(int valor) {  
        this.valor = valor;  
        this.esquerda = null;  
        this.direita = null;  
    }  
}
```

Assim, cada nó possui três informações principais:

- seu valor;
- uma referência para o filho esquerdo;
- uma referência para o filho direito.

Podemos representar um nó da seguinte forma:

No = (valor , esquerda , direita)

1.2 Construtor

O construtor da classe é:

```
public ArvoreBinariaBusca() {  
    this.raiz = null;  
}
```

Quando uma nova árvore é criada, ela começa vazia:

```
raiz = null
```

Por exemplo:

```
ArvoreBinariaBusca arvore =  
new ArvoreBinariaBusca();
```

Nesse momento não existem nós na árvore.

1.3 Inserção

A inserção é realizada pelo método:

```
public void inserir(int valor) {  
    raiz = inserirRecursivo(raiz, valor);  
}
```

Esse método chama o método auxiliar recursivo:

```
private No inserirRecursivo(No no, int valor)
```

A ideia da inserção é comparar o novo valor com o valor do nó atual.

1.3.1 Posição vazia

Caso o nó atual seja null:

```
if (no == null) {  
    return new No(valor);  
}
```

isso significa que encontramos a posição onde o novo elemento deverá ser inserido.

Portanto:

```
null novo
```

```
No(valor)
```

1.3.2 Valor menor

Se:

`valor < no.valor`

o elemento deve ser inserido na subárvore esquerda:

```
if (valor < no.valor) {  
no.esquerda =  
inserirRecursivo(no.esquerda, valor);  
}
```

1.3.3 Valor maior

Se:

`valor > no.valor`

o elemento deve ser inserido na subárvore direita:

```
else if (valor > no.valor) {  
no.direita =  
inserirRecursivo(no.direita, valor);  
}
```

1.3.4 Valor duplicado

Caso:

`valor == no.valor`

o programa não cria um novo nó:

```
else {  
System.out.println(  
"O valor ja existe na arvore."  
);  
}
```

Portanto, a implementação não permite valores duplicados.

1.4 Busca Recursiva

A busca é realizada pelo método:

```
public boolean buscar(int valor) {  
    return buscarRecursivo(raiz, valor);  
}
```

O método auxiliar é:

```
private boolean buscarRecursivo(No no, int valor)
```

A primeira condição verifica se chegamos a uma posição vazia:

```
if (no == null) return false;
```

Nesse caso, o valor não está presente.

Depois verificamos:

```
if (no.valor == valor) return true;
```

Se o valor do nó for igual ao valor procurado, o elemento foi encontrado.

Caso contrário, utilizamos a propriedade da BST.

Se:

$$valor < no.valor$$

a busca continua na esquerda:

```
return buscarRecursivo(no.esquerda, valor);
```

Caso contrário, continua na direita:

```
return buscarRecursivo(no.direita, valor);
```

Por exemplo, para procurar 40 :

$$40 < 50$$

portanto:

$$50 \rightarrow 30$$

Depois:

$$40 > 30$$

portanto:

$30 \rightarrow 40$

Finalmente:

$40 = 40$

Logo, o resultado é:

true

1.5 Busca Iterativa

A classe também possui uma versão iterativa:

```
public boolean buscarIterativo(int valor)
```

Nesse caso, não é utilizada recursão. É utilizado um laço while.

O processo começa na raiz:

```
No atual = raiz;
```

Enquanto o nó atual não for nulo:

```
while (atual != null)
```

o programa verifica o valor.

Se:

```
a t u a l . v a l o r = v a l o r
```

retorna:

true

Se:

```
v a l o r < a t u a l . v a l o r
```

avança para:

```
a t u a l = a t u a l . e s q u e r d a
```

Caso contrário:

```
a t u a l = a t u a l . d i r e i t a
```

Quando o algoritmo chega a null, significa que o elemento não foi encontrado:

false

1.6 Percursos da Árvore

Um percurso consiste em visitar todos os nós da árvore seguindo uma determinada ordem.

A classe implementa três percursos recursivos principais:

1. Pré-ordem;
2. Em ordem;
3. Pós-ordem.

Além disso, implementa:

1. BFS – Busca em Largura;
2. DFS – Busca em Profundidade.

1.7 Pré-ordem

A pré-ordem segue a sequência:

Raiz $\rightarrow$ Esquerda $\rightarrow$ Direita

O código é:

```
private void preOrdemRecursivo(No no) {  
    if (no != null) {  
        System.out.print(no.valor + " ");  
        preOrdemRecursivo(no.esquerda);  
        preOrdemRecursivo(no.direita);  
    }  
}
```

Para a árvore:

```
50  
/  
30 70  
/ \ /  
20 40 60 80
```

o resultado é:

50

30

20

40

70

60

80

A característica principal é que a raiz é processada primeiro.

1.8 Em ordem

O percurso em ordem segue:

Esquerda → Raiz → Direita

O código é:

```
private void emOrdemRecursoivo(No no) {  
    if (no != null) {  
        emOrdemRecursoivo(no.esquerda);  
        System.out.print(no.valor + " ");  
        emOrdemRecursoivo(no.direita);  
    }  
}
```

Para a árvore apresentada:

20

30

40

50

60

70

80

Uma propriedade importante das BSTs é:

Percurso em ordem valores em ordem crescente

Isso ocorre porque os menores valores estão sempre à esquerda e os maiores à direita.

1.9 Pós-ordem

O percurso pós-ordem segue:

Esquerda → Direita → Raiz

O código é:

```
private void posOrdemRecursivo(No no) {  
    if (no != null) {  
        posOrdemRecursivo(no.esquerda);  
        posOrdemRecursivo(no.direita);  
        System.out.print(no.valor + " ");  
    }  
}
```

O resultado será:

20

40

30

60

80

70

50

Nesse percurso, a raiz é processada por último.

1.10 BFS – Busca em Largura

BFS significa *Breadth-First Search*, ou Busca em Largura.

A ideia é visitar a árvore nível por nível:

$n_1 \text{ vel } 0 \rightarrow n_1 \text{ vel } 1 \rightarrow n_1 \text{ vel } 2 \rightarrow$

A implementação utiliza uma fila:

```
Queue<No> fila = new LinkedList<>();
```

Inicialmente, a raiz é adicionada:

```
fila.add(raiz);
```

Depois, o primeiro elemento é retirado:

```
No atual = fila.poll();
```

Os filhos são então adicionados à fila.

A fila segue o princípio:

FIFO

First In, First Out.

Para a árvore:

```
50
/
30 70
/ \ /
20 40 60 80
```

o resultado é:

```
50
30
70
20
40
60
80
```

1.11 DFS – Busca em Profundidade

DFS significa *Depth-First Search*, ou Busca em Profundidade.

Nesse método, utilizamos uma pilha:

```
Stack<No> pilha = new Stack<>();
```

A raiz é colocada na pilha:

```
pilha.push(raiz);
```

Depois, o elemento do topo é retirado:

```
No atual = pilha.pop();
```

A pilha segue o princípio:

LIFO

Last In, First Out.

Por isso, o código coloca primeiro o filho direito:

```
if (atual.direita != null)
    pilha.push(atual.direita);
```

e depois o filho esquerdo:

```
if (atual.esquerda != null)
    pilha.push(atual.esquerda);
```

Como a esquerda é colocada por último, ela fica no topo da pilha e será processada primeiro.

Assim, nessa implementação, o DFS produz a mesma ordem do percurso pré-ordem:

Raiz → Esquerda → Direita

Resultado:

50

30

20

40

70

60

80

1.12 Altura da Árvore

A altura é calculada pelo método:

```
private int calcularAlturaRecursivo(No no)
```

A implementação utiliza a seguinte convenção:

$H(\text{a rvore vazia}) = 1$

Consequentemente:

$H(\text{folha}) = 0$

A fórmula utilizada é:

$H(n) = 1 + \max(H(n \text{ esquerda}), H(n \text{ direita}))$

No código:

```
return 1 + Math.max(
    calcularAlturaRecursivo(no.esquerda),
    calcularAlturaRecursivo(no.direita)
);
```

Considere:

```
50
/
30 70
/
20 40
```

As folhas possuem altura:

$H(20) = 0$

$H(40) = 0$

$H(70) = 0$

Para o nó 30 :

$H(30) = 1 + \max(0, 0) = 1$

Para a raiz:

$H(50) = 1 + \max(1, 0) = 2$

Logo:

$H = 2$

1.13 Visualização da Estrutura

O método:

```
public void mostrarEstrutura()
```

é utilizado para apresentar visualmente a organização da árvore.

Uma possível saída é:

```
--- Estrutura Hierarquica ---
(R) 50
|--(E) 30
| |--(E) 20
| --(D) 40 --(D) 70
|--(E) 60
'--(D) 80
```

Os símbolos representam:

- (R): raiz;
- (E): filho esquerdo;
- (D): filho direito.

O método utiliza recursão para percorrer todos os nós e construir visualmente a hierarquia.

1.14 Conclusão

A classe `ArvoreBinariaBusca` implementa uma estrutura de dados baseada na organização hierárquica dos valores.

A principal propriedade é:

Esquerda < Raiz < Direita

Essa propriedade é utilizada tanto na inserção quanto na busca.

Os principais conceitos implementados na classe são:

- **Recursão**, utilizada na inserção, busca, percursos e cálculo da altura;
- **Fila** (Queue), utilizada no BFS;
- **Pilha** (Stack), utilizada no DFS;
- **Percursos em árvores**, como pré-ordem, em ordem e pós-ordem;
- **Altura da árvore**;
- **Estrutura hierárquica de nós**.

Para a árvore:

```

50
/
30 70
/ \ /
20 40 60 80

```

temos:

Pre -ordem = 50 ,

30 ,

20 ,

40 ,

70 ,

60 ,

80 Em ordem = 20 ,

30 ,

40 ,

50 ,

60 ,

70 ,

80 P o s-ordem = 20 ,

40 ,

30 ,

60 ,

80 ,
70 ,
50 BFS = 50 ,
30 ,
70 ,
20 ,
40 ,
60 ,
80 DFS = 50 ,
30 ,
20 ,
40 ,
70 ,
60 ,
80 Altura = 2

Portanto, a classe demonstra na prática os principais conceitos relacionados a uma **Árvore Binária de Busca**, incluindo inserção, pesquisa, diferentes estratégias de percurso e cálculo da altura.

1.15 Visualização Do Código

```
package model;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

2 usages  📄 victorhugoEngineer
public class ArvoreBinariaBusca {
    21 usages
    private No raiz;

    1 usage  📄 victorhugoEngineer
    public ArvoreBinariaBusca() {
        this.raiz = null;
    }

    // Inserção mantendo as propriedades da BST (Recursivo)
    1 usage  📄 victorhugoEngineer
    public void inserir(int valor) {
        raiz = inserirRecursivo(raiz, valor);
    }

    3 usages  📄 victorhugoEngineer
    private No inserirRecursivo(No no, int valor) {
        if (no == null) {
            return new No(valor);
        }
        if (valor < no.valor) {
            no.esquerda = inserirRecursivo(no.esquerda, valor);
        } else if (valor > no.valor) {
            no.direita = inserirRecursivo(no.direita, valor);
        } else {
            System.out.println("> O valor " + valor + " ja existe na arvore (duplicados ignorados).");
        }
        return no;
    }

    // Busca Recursiva
    1 usage  📄 victorhugoEngineer
    public boolean buscar(int valor) {
        return buscarRecursivo(raiz, valor);
    }
}
```

```
// Busca Recursiva
1 usage  📄 victorhugoEngineer
public boolean buscar(int valor) {
    return buscarRecursivo(raiz, valor);
}

3 usages  📄 victorhugoEngineer
private boolean buscarRecursivo(No no, int valor) {
    if (no == null) return false;
    if (no.valor == valor) return true;
    if (valor < no.valor) return buscarRecursivo(no.esquerda, valor);
    return buscarRecursivo(no.direita, valor);
}

// Busca Iterativa (Alternativa)
📄 victorhugoEngineer
public boolean buscarIterativo(int valor) {
    No atual = raiz;
    while (atual != null) {
        if (atual.valor == valor) return true;
        if (valor < atual.valor) atual = atual.esquerda;
        else atual = atual.direita;
    }
    return false;
}

// Percurso Pré-ordem: Raiz → Esquerda → Direita
1 usage  📄 victorhugoEngineer
public void mostrarPreOrdem() {
    if (raiz == null) {
        System.out.println("Arvore vazia.");
    } else {
        preOrdemRecursivo(raiz);
        System.out.println();
    }
}

3 usages  📄 victorhugoEngineer
private void preOrdemRecursivo(No no) {
    if (no != null) {
```

```
}

3 usages victorhugoEngineer
private void preOrdemRecursivo(No no) {
    if (no != null) {
        System.out.print(no.valor + " ");
        preOrdemRecursivo(no.esquerda);
        preOrdemRecursivo(no.direita);
    }
}

// Percurso Em Ordem: Esquerda → Raiz → Direita
1 usage victorhugoEngineer
public void mostrarEmOrdem() {
    if (raiz == null) {
        System.out.println("Árvore vazia.");
    } else {
        emOrdemRecursivo(raiz);
        System.out.println();
    }
}

3 usages victorhugoEngineer
private void emOrdemRecursivo(No no) {
    if (no != null) {
        emOrdemRecursivo(no.esquerda);
        System.out.print(no.valor + " ");
        emOrdemRecursivo(no.direita);
    }
}

// Percurso Pós-ordem: Esquerda → Direita → Raiz
1 usage victorhugoEngineer
public void mostrarPosOrdem() {
    if (raiz == null) {
        System.out.println("Árvore vazia.");
    } else {
        posOrdemRecursivo(raiz);
        System.out.println();
    }
}
}
```

```
    } else {  
        posOrdemRecursivo(raiz);  
        System.out.println();  
    }  
}  
  
3 usages  victorhugoEngineer  
private void posOrdemRecursivo(No no) {  
    if (no != null) {  
        posOrdemRecursivo(no.esquerda);  
        posOrdemRecursivo(no.direita);  
        System.out.print(no.valor + " ");  
    }  
}  
  
// BFS - Busca em Largura (Utilizando Fila - Queue)  
1 usage  victorhugoEngineer  
public void mostrarBFS() {  
    if (raiz == null) {  
        System.out.println("Arvore vazia.");  
        return;  
    }  
    Queue<No> fila = new LinkedList<>();  
    fila.add(raiz);  
  
    while (!fila.isEmpty()) {  
        No atual = fila.poll();  
        System.out.print(atual.valor + " ");  
  
        if (atual.esquerda != null) fila.add(atual.esquerda);  
        if (atual.direita != null) fila.add(atual.direita);  
    }  
    System.out.println();  
}  
  
// DFS - Busca em Profundidade (Utilizando Pilha - Stack)  
1 usage  victorhugoEngineer  
public void mostrarDFS() {  
    if (raiz == null) {  
        System.out.println("Arvore vazia.");  
        return;  
    }  
}
```

```

1 usage  victorhugoEngineer
public void mostrarDFS() {
    if (raiz == null) {
        System.out.println("Árvore vazia.");
        return;
    }
    Stack<No> pilha = new Stack<>();
    pilha.push(raiz);

    while (!pilha.isEmpty()) {
        No atual = pilha.pop();
        System.out.print(atual.valor + " ");

        // Empilha o filho direito primeiro para processar a esquerda antes
        if (atual.direita != null) pilha.push(atual.direita);
        if (atual.esquerda != null) pilha.push(atual.esquerda);
    }
    System.out.println();
}

// Solução Recursiva para calcular a Altura da Árvore
// Convenção: árvore de 1 nó tem altura 0; árvore vazia tem altura -1.
1 usage  victorhugoEngineer
public void mostrarAltura() {
    if (raiz == null) {
        System.out.println("Árvore vazia (Altura: -1)");
    } else {
        System.out.println("Altura da árvore: " + calcularAlturaRecursivo(raiz));
    }
}

3 usages  victorhugoEngineer
private int calcularAlturaRecursivo(No no) {
    if (no == null) {
        return -1;
    }
    return 1 + Math.max(calcularAlturaRecursivo(no.esquerda), calcularAlturaRecursivo(no.direita));
}

// Visualização da estrutura hierárquica identificando Esquerda (E) e Direita (D)
1 usage  victorhugoEngineer

```

```
3 usages  victorhugoEngineer
private int calcularAlturaRecursivo(No no) {
    if (no == null) {
        return -1;
    }
    return 1 + Math.max(calcularAlturaRecursivo(no.esquerda), calcularAlturaRecursivo(no.direita));
}

// Visualização da estrutura hierárquica identificando Esquerda (E) e Direita (D)
1 usage  victorhugoEngineer
public void mostrarEstrutura() {
    if (raiz == null) {
        System.out.println("Árvore vazia.");
        return;
    }
    System.out.println("\n--- Estrutura Hierarquica ---");
    System.out.println("(R) " + raiz.valor);
    exibirEstruturaRecursivo(raiz.esquerda, prefixo: " ", ehEsquerda: true);
    exibirEstruturaRecursivo(raiz.direita, prefixo: " ", ehEsquerda: false);
    System.out.println("-----");
}

4 usages  victorhugoEngineer
private void exibirEstruturaRecursivo(No no, String prefixo, boolean ehEsquerda) {
    if (no != null) {
        System.out.print(prefixo);
        System.out.print(ehEsquerda ? "|—(E) " : "|--(D) ");
        System.out.println(no.valor);

        exibirEstruturaRecursivo(no.esquerda, prefixo: prefixo + (ehEsquerda ? "| " : "| "), ehEsquerda: true);
        exibirEstruturaRecursivo(no.direita, prefixo: prefixo + (ehEsquerda ? "| " : "| "), ehEsquerda: false);
    }
}
```


2 Classe No

```
package model;

@ victorhugoEngineer
public class No {
    14 usages
    int valor;
    15 usages
    No esquerda;
    15 usages
    No direita;

    1 usage @ victorhugoEngineer
    public No(int valor) {
        this.valor = valor;
        this.esquerda = null;
        this.direita = null;
    }
}
```

A classe No representa a estrutura básica utilizada para construir uma Árvore Binária de Busca. Cada objeto dessa classe representa um elemento, ou nó, da árvore.

A implementação é apresentada a seguir:

```
public class No {
    int valor;
    No esquerda;
    No direita;

    public No(int valor) {
        this.valor = valor;
```

```
this.esquerda = null;  
this.direita = null;  
}  
  
}
```

Cada nó possui três informações principais:

- o valor armazenado;
- uma referência para o filho esquerdo;
- uma referência para o filho direito.

Matematicamente, podemos representar um nó como:

```
No = ( valor ,  
      esquerda ,  
      direita )
```

2.0.1 Declaração da classe

A classe é declarada através de:

```
public class No
```

A palavra-chave `class` indica que estamos criando uma nova classe em Java. A palavra-chave `public` permite que a classe seja utilizada por outras classes do programa.

Cada objeto criado a partir dessa classe representará um nó da árvore.

Por exemplo:

```
No no = new No(50);
```

Nesse caso, é criado um nó contendo o valor 50 .

Podemos representá-lo da seguinte maneira:

No 50

2.0.2 Atributo valor

O primeiro atributo é:

```
int valor;
```

Esse atributo armazena o valor que pertence ao nó.

Por exemplo:

```
No no = new No(50);
```

Após a criação do objeto:

```
no.valor = 50
```

Portanto, podemos representar o nó como:

```
valor = 50
```

Esse valor será utilizado pela Árvore Binária de Busca para determinar se outros elementos devem ficar à esquerda ou à direita.

2.0.3 Atributo esquerda

O segundo atributo é:

```
No esquerda;
```

Esse atributo armazena uma referência para outro objeto da classe No.

Essa referência representa o **filho esquerdo** do nó atual.

Por exemplo:

```
No no = new No(50);
no.esquerda = new No(30);
```

A estrutura será:

```
50
/
30
```

Ou seja:

50. esquerda $\rightarrow$ 30

Caso o nó não possua um filho esquerdo, essa referência será:

esquerda = null

2.0.4 Atributo direita

O terceiro atributo é:

No direita;

Assim como esquerda, esse atributo armazena uma referência para outro objeto da classe No.

Nesse caso, a referência representa o **filho direito**.

Por exemplo:

```
No no = new No(50);  
no.direita = new No(70);
```

A estrutura será:

50

70

Podemos representar essa relação como:

50. direita $\rightarrow$ 70

Caso não exista um filho direito:

direita = null

2.1 Construtor da Classe

O construtor da classe é:

```
public No(int valor) {  
    this.valor = valor;  
    this.esquerda = null;
```

```
this.direita = null;  
}
```

O construtor é executado sempre que um novo objeto da classe No é criado.

Por exemplo:

```
No no = new No(50);
```

Nesse caso, o valor 50 é passado como parâmetro para o construtor.

2.1.1 this.valor = valor

A instrução:

```
this.valor = valor;
```

atribui o valor recebido pelo construtor ao atributo do objeto.

Existem dois elementos chamados valor:

- `this.valor`: atributo pertencente ao objeto;
- `valor`: parâmetro recebido pelo construtor.

Assim, quando executamos:

```
No no = new No(50);
```

temos:

```
n o . v a l o r = 50
```

A palavra-chave `this` indica que estamos nos referindo ao atributo pertencente ao objeto atual.

2.1.2 this.esquerda = null

A instrução:

```
this.esquerda = null;
```

informa que, inicialmente, o novo nó não possui um filho esquerdo.

Portanto:

```
n o . e s q u e r d a = null
```

Isso significa que ainda não existe outro nó conectado à esquerda.

2.1.3 this.direita = null

Da mesma forma:

```
this.direita = null;
```

indica que inicialmente não existe um filho direito.

Portanto:

```
n o . d i r e i t a = null
```

2.2 Representação de um Novo Nó

Logo após a execução:

```
No no = new No(50);
```

temos:

```
+-----+
| 50 |
+-----+
/
/
null null
esquerda direita
```

Podemos representar matematicamente como:

```
N o = ( 50 , null , null )
```

Isso significa que o nó possui o valor 50 , mas ainda não possui filhos.

2.3 Conectando Nós

Os nós podem ser conectados uns aos outros através dos atributos esquerda e direita.

Por exemplo:

```
No raiz = new No(50);  
No esquerda = new No(30);  
No direita = new No(70);
```

```
raiz.esquerda = esquerda;  
raiz.direita = direita;
```

A estrutura resultante será:

```
50  
/  
30 70
```

Nesse caso:

```
raiz.valor = 50  
raiz.esquerda.valor = 30  
raiz.direita.valor = 70
```

Podemos representar a árvore como:

50 esquerda → 30 direita → 70

2.4 Relação com a Classe ArvoreBinariaBusca

A classe No funciona em conjunto com a classe ArvoreBinariaBusca.

Enquanto a classe No representa cada elemento individual, a classe ArvoreBinariaBusca é responsável por organizar esses elementos.

Podemos representar essa relação da seguinte maneira:

No → representa um elemento

ArvoreBinariaBusca → organiza os elementos

Na classe ArvoreBinariaBusca, temos:

```
private No raiz;
```

A variável `raiz` armazena uma referência para o primeiro nó da árvore.

Por exemplo:

```
ArvoreBinariaBusca arvore =
new ArvoreBinariaBusca();
```

```
arvore.inserir(50);
arvore.inserir(30);
arvore.inserir(70);
```

A estrutura será:

```
raiz
|
v
50
/
30 70
```

Assim, a `ArvoreBinariaBusca` possui uma referência para um objeto `No`, e esse objeto possui referências para outros objetos `No`.

2.5 Estrutura Completa

Considere a inserção dos valores:

```
50 ,
30 ,
70 ,
20 ,
40 ,
60 ,
80
```

A estrutura será:

```
50
/
```

```

30 70
/ \ /
20 40 60 80

```

Cada elemento dessa estrutura é um objeto da classe No.

Podemos observar as relações:

50. esquerda $\rightarrow$ 30

50. direita $\rightarrow$ 70

30. esquerda $\rightarrow$ 20

30. direita $\rightarrow$ 40

70. esquerda $\rightarrow$ 60

70. direita $\rightarrow$ 80

Os nós 20, 40, 60 e 80 são folhas, pois não possuem filhos:

20. esquerda = null

20. direita = null

e o mesmo ocorre para os demais nós folhas.

2.6 Resumo

A classe No é responsável por representar cada elemento da Árvore Binária de Busca.

Cada objeto possui:

valor refer e ^ ncia para esquerda refer e ^ ncia para direita

Podemos resumir sua estrutura como:

No = (valor dado, esquerda filho esquerdo, direita filho direito)

O atributo valor armazena o dado, enquanto esquerda e direita permitem conectar o nó a outros nós.

Dessa forma, vários objetos da classe No podem ser conectados para formar uma árvore:

V a rios objetos No A rvore Bin a ria de Busca

Portanto, a classe No pode ser entendida como o **bloco de construção fundamental** da Árvore Binária de Busca. Sem ela, a classe ArvoreBinariaBusca não teria os objetos necessários para armazenar e organizar os valores.

3 Classe Main

A classe `Main` é responsável por executar o programa da Árvore Binária de Busca. Ela funciona como a interface de interação entre o usuário e a classe `ArvoreBinariaBusca`.

Por meio dessa classe, o usuário pode:

- inserir valores na árvore;
- buscar valores;
- visualizar diferentes percursos;
- visualizar a árvore utilizando BFS;
- visualizar a árvore utilizando DFS;
- calcular a altura da árvore;
- visualizar a estrutura hierárquica;
- encerrar o programa.

A classe utiliza um menu interativo para permitir que o usuário escolha a operação desejada.

3.1 Método `main`

O método principal do programa é:

```
public static void main(String[] args)
```

O método `main` é o ponto de entrada da aplicação Java. Quando o programa é executado, a execução começa nesse método.

A estrutura básica é:

```
public static void main(String[] args) {  
    ...  
}
```

O parâmetro:

```
String[] args
```

permite receber argumentos passados pela linha de comando. Neste programa, entretanto, esse parâmetro não é utilizado.

3.2 Criação da Árvore

A primeira operação realizada é:

```
ArvoreBinariaBusca arvore =  
new ArvoreBinariaBusca();
```

Essa instrução cria um novo objeto da classe `ArvoreBinariaBusca`.

Inicialmente, a árvore está vazia:

```
raiz = null
```

A variável `arvore` será utilizada posteriormente para acessar todos os métodos da árvore.

Por exemplo:

```
arvore.inserir(50);  
arvore.buscar(50);  
arvore.mostrarEmOrdem();
```

3.3 Criação do Scanner

O programa utiliza a classe `Scanner` para receber dados digitados pelo usuário:

```
Scanner scanner = new Scanner(System.in);
```

O objeto `scanner` permite ler informações do teclado.

Por exemplo:

```
int valor = scanner.nextInt();
```

Nesse caso, o programa espera que o usuário digite um número inteiro.

A entrada utilizada é:

```
System.in
```

que representa a entrada padrão do sistema, normalmente o teclado.

3.4 Variável de Opção

O programa cria uma variável para armazenar a opção escolhida pelo usuário:

```
int opcao = 0;
```

Essa variável controla qual operação será executada no menu.

Por exemplo:

Opção 1 Inserir valor 2 Buscar valor 3 Pré-ordem 4 Em ordem 5 Pós-ordem 6 BFS 7 DFS 8 Altura 9 Estrutura 10 Sair

3.5 Estrutura do-while

O menu é executado dentro de um laço:

```
do {
    ...
} while (opcao != 10);
```

O do-while garante que o menu seja exibido pelo menos uma vez.

A condição:

opcao

!=

10

significa que o programa continuará executando enquanto a opção escolhida for diferente de 10.

Quando o usuário escolher:

10

o programa será encerrado.

Podemos representar o funcionamento como:

Exibir menu → Ler opção → Executar operação → Voltar ao menu

Esse ciclo continua até que:

opcao = 10

3.6 Verificação da Entrada

Antes de ler a opção, o programa verifica se o usuário realmente digitou um número inteiro:

```
if (scanner.hasNextInt()) {  
    opcao = scanner.nextInt();  
}
```

O método:

```
scanner.hasNextInt()
```

retorna true caso exista um número inteiro disponível para leitura.

Caso contrário, retorna false.

Se o usuário digitar, por exemplo:

```
abc
```

o programa executará:

```
System.out.println(  
    "Por favor, digite um numero valido."  
);
```

Em seguida:

```
scanner.next();
```

remove a entrada inválida para que o programa possa continuar funcionando normalmente.

3.7 Estrutura switch

Depois que a opção é lida, o programa utiliza:

```
switch (opcao)
```

O switch permite executar diferentes blocos de código dependendo do valor da variável `opcao`.

A estrutura pode ser representada como:

`opcao` → 1 Inserir 2 Buscar 3 Pre-ordem 4 Em ordem 5 Pos-ordem 6 BFS 7 DFS 8 Altura 9 Estrutura 10 Sair

3.8 Opção 1 – Inserir Valor

```

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 1
Digite o valor a ser inserido: 50
Valor 50 inserido.

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: |

```

Figura 1 – Insire o Valor

O primeiro caso é:

```
case 1:
System.out.print(
"Digite o valor a ser inserido: "
);

int valInserir = scanner.nextInt();

arvore.inserir(valInserir);

System.out.println(
    "Valor " + valInserir + " inserido."
);
break;
```

Primeiro, o programa solicita um valor ao usuário.

Depois:

```
int valInserir = scanner.nextInt();
```

lê o número digitado.

Em seguida, o valor é enviado para a árvore:

```
arvore.inserir(valInserir);
```

Esse método pertence à classe *ArvoreBinariaBusca*.

Por exemplo, se o usuário digitar:

50

será executado:

```
arvore.inserir(50);
```

A árvore passará a possuir o nó 50 .

3.9 Opção 2 – Buscar Valor

O segundo caso realiza uma busca:

```

Digite o valor para buscar: 2
>> AVISO: O valor 2 NAO FOI ENCONTRADO na arvore.

=====
                ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 2
Digite o valor para buscar: 50
>> SUCESSO: O valor 50 FOI ENCONTRADO na arvore.

=====
                ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: |

```

Figura 2 – Busca De Valor

```

case 2:
System.out.print(
"Digite o valor para buscar: "
);

int valBuscar = scanner.nextInt();

if (arvore.buscar(valBuscar)) {
    ...
} else {
    ...
}
break;

```

O valor procurado é armazenado em:

```
int valBuscar
```

Depois, o programa chama:

```
arvore.buscar(valBuscar)
```

Esse método retorna um valor booleano:

true ou false

Se o valor for encontrado:

```
if (arvore.buscar(valBuscar))
```

o programa informa que a busca teve sucesso.

Caso contrário:

```
else
```

informa que o valor não foi encontrado.

3.10 Opção 3 – Pré-ordem

A opção 3 executa:

```
arvore.mostrarPreOrdem();
```

Esse método realiza o percurso:

Raiz → Esquerda → Direita

Para uma árvore como:

```
50
 /
30 70
```

o resultado será:

50

5

7

```

Escolha uma opcao: 1
Digite o valor a ser inserido: 7
Valor 7 inserido.

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

=====

Escolha uma opcao: 3
Percurso Pre-ordem: 50 5 7

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

=====

Escolha uma opcao: |

```

Figura 3 – PréOrdem

3.11 Opção 4 – Em Ordem

A opção 4 chama:

```
arvore.mostrarEmOrdem();
```

Esse percurso segue:

Esquerda → Raiz → Direita

Em uma BST, esse percurso apresenta os valores em ordem crescente.

```

Escolha uma opcao: 1
Digite o valor a ser inserido: 7
Valor 7 inserido.

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

=====

Escolha uma opcao: 3
Percurso Pre-ordem: 50 5 7

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

=====

Escolha uma opcao: |

```

Figura 4 – Valor em ordem

3.12 Opção 5 – Pós-ordem

A opção 5 executa:

```
arvore.mostrarPosOrdem();
```

A ordem de visita é:

Esquerda → Direita → Raiz

Assim, a raiz é processada por último.

```
=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 5
Percurso Pos-ordem: 7 5 50

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao:
```

Figura 5 – Valor em Pós ordem

3.13 Opção 6 – BFS

A opção 6 chama:

```
arvore.mostrarBFS();
```

BFS significa *Breadth-First Search*, ou Busca em Largura.

O percurso é realizado nível por nível.

```

Escolha uma opcao: 5
Percurso Pos-ordem: 7 5 50

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 6
Percurso BFS (Busca em Largura): 50 5 7

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao:

```

Figura 6 – BFS

3.14 Opção 7 – DFS

A opção 7 executa:

```
arvore.mostrarDFS();
```

DFS significa *Depth-First Search*, ou Busca em Profundidade.

Na implementação apresentada, é utilizada uma pilha para realizar o percurso.

O resultado produzido é equivalente ao percurso pré-ordem:

Raiz → Esquerda → Direita

```

Escolha uma opcao: 6
Percurso BFS (Busca em Largura): 50 5 7

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 7
Percurso DFS (Busca em Profundidade): 50 5 7

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao:

```

Figura 7 – DFS

3.15 Opção 8 – Altura

A opção 8 executa:

```
arvore.mostrarAltura();
```

Esse método calcula a altura da árvore.

```

ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 8
Altura da arvore: 2

=====

ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: |

```

Figura 8 – Altura

A implementação utiliza a convenção:

$H(\text{a rvore vazia}) = 1$

e:

$H(\text{folha}) = 0$

A fórmula utilizada é:

$H(n) = 1 + \max(H(n \text{ esquerda}), H(n \text{ direita}))$

3.16 Opção 9 – Estrutura da Árvore

```

ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 9

--- Estrutura Hierarquica ---
(R) 50
    ├──(E) 5
    |   |--(D) 7
=====

ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: |

```

Figura 9 – Estrutura Da Arvore

A opção 9 executa:

```
arvore.mostrarEstrutura();
```

Esse método apresenta a estrutura hierárquica da árvore.

Por exemplo:

--- Estrutura Hierarquica ---

```
(R) 50
|--(E) 30
| |--(E) 20
| --(D) 40 --(D) 70
|--(E) 60
'--(D) 80
```

Os símbolos representam:

- (R): raiz;
- (E): esquerda;
- (D): direita.

3.17 Opção 10 – Encerrar

```
ARVORE BINARIA DE BUSCA (BST)

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 10
Encerrando o programa ...

Process finished with exit code 0
|
```

Figura 10 – Encerrando o programa

A opção 10 executa:

```
System.out.println(
```

```
"Encerrando o programa..."  
);
```

Depois disso, o switch termina.

Na próxima verificação do do-while:

opcao

!=

10

será avaliado como:

10 10

que é:

false

Consequentemente, o laço é encerrado.

3.18 Opção Inválida

Caso o usuário digite um número que não esteja entre 1 e 10 , será executado:

```
default:  
System.out.println(  
"Opcao invalida! Tente novamente."  
);  
break;
```

Por exemplo, se o usuário digitar:

15

o programa informa que a opção é inválida e volta para o menu.

3.19 Fechamento do Scanner

Depois que o usuário escolhe a opção 10 e o laço termina, o programa executa:

```
scanner.close();
```

Essa instrução fecha o objeto Scanner e libera o recurso utilizado para leitura da entrada.

```
=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: 15
Opcao invalida! Tente novamente.

=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao:
```

Figura 11 – Opção invalida

3.20 Método `exibirMenu`

A classe possui ainda um método auxiliar:

```
private static void exibirMenu()
```

Esse método é responsável apenas por exibir as opções disponíveis para o usuário.

Ele contém vários comandos:

```
System.out.println(...)
```

que imprimem o menu no console.

O menu apresentado é:

ARVORE BINARIA DE BUSCA (BST)

=====

Inserir valor

Buscar valor

Mostrar Pre-ordem

Mostrar Em ordem

Mostrar Pos-ordem

Mostrar BFS - Busca em Largura

Mostrar DFS - Busca em Profundidade

Mostrar altura da arvore

Mostrar estrutura da arvore

Sair

Escolha uma opcao:

O método é declarado como `private` porque ele é utilizado somente dentro da própria classe `Main`.

A palavra-chave `static` permite que o método seja chamado diretamente dentro do método `main`, sem a necessidade de criar um objeto da classe `Main`.

Assim:

```
exibirMenu();
```

pode ser chamado diretamente.

3.21 Fluxo Geral do Programa

O funcionamento completo da classe Main pode ser representado da seguinte forma:

Início ↓ Criar a árvore ↓ Criar Scanner ↓ Exibir menu ↓ Ler opção a ~ o ↓ Executar operação escolhida ↓ Operação = 10? Não → Voltar ao menu Sim → Encerrar programa

Dessa forma, o programa permanece executando até que o usuário escolha a opção 10.

3.22 Relação entre as Classes

O projeto possui três responsabilidades principais:

Classe	Responsabilidade
Representar cada nó da árvore	heightNo
hline ArvoreBinariaBusca	Organizar e manipular a árvore
Interagir com o usuário e executar o programa	heightMain

Podemos visualizar a relação entre elas:

```

Main
|
| utiliza
v
ArvoreBinariaBusca
|
| possui
v
No
/
esquerda direita
| |
v v
No No

```

Portanto, cada classe possui uma função específica.

A classe No representa os elementos individuais da árvore. A classe ArvoreBinariaBusca implementa as operações sobre esses elementos, como inserção, busca e percursos. Por fim, a classe Main fornece a interação com o usuário por meio do menu.

3.23 Conclusão

A classe Main funciona como o ponto de controle do programa. Ela não implementa diretamente a lógica da Árvore Binária de Busca; em vez disso, utiliza os métodos fornecidos pela classe ArvoreBinariaBusca.

O funcionamento pode ser resumido em:

Usuário → Main → ArvoreBinariaBusca → Nó

O usuário escolhe uma operação no menu, a classe Main interpreta essa escolha e chama o método correspondente da árvore.

Dessa forma, existe uma separação clara de responsabilidades:

Nó → estrutura dos dados ArvoreBinariaBusca → lógica da árvore Main → interação com o usuário

Essa organização torna o programa mais fácil de compreender, testar e modificar.

```
package model;

import java.util.Scanner;

@victorhugoEngineer
public class Main {
    @victorhugoEngineer
    public static void main(String[] args) {
        ArvoreBinariaBusca arvore = new ArvoreBinariaBusca();
        Scanner scanner = new Scanner(System.in);
        int opcao = 0;

        do {
            exibirMenu();
            if (scanner.hasNextInt()) {
                opcao = scanner.nextInt();
            } else {
                System.out.println("Por favor, digite um numero valido.");
                scanner.next(); // Limpa a entrada inválida
                continue;
            }

            switch (opcao) {
                case 1:
                    System.out.print("Digite o valor a ser inserido: ");
                    int valInserir = scanner.nextInt();
                    arvore.inserir(valInserir);
                    System.out.println("Valor " + valInserir + " inserido.");
                    break;

                case 2:
                    System.out.print("Digite o valor para buscar: ");
                    int valBuscar = scanner.nextInt();
                    if (arvore.buscar(valBuscar)) {
                        System.out.println(">> SUCESSO: O valor " + valBuscar + " FOI ENCONTRADO na arvore.");
                    } else {
                        System.out.println(">> AVISO: O valor " + valBuscar + " NAO FOI ENCONTRADO na arvore.");
                    }
                    break;

                case 3:
```

```
        System.out.println(">> AVISO: O valor " + valBuscar + " NAO FOI ENCONTRADO na arvore.");
    }
    break;

    case 3:
        System.out.print("Percurso Pre-ordem: ");
        arvore.mostrarPreOrdem();
        break;

    case 4:
        System.out.print("Percurso Em ordem: ");
        arvore.mostrarEmOrdem();
        break;

    case 5:
        System.out.print("Percurso Pos-ordem: ");
        arvore.mostrarPosOrdem();
        break;

    case 6:
        System.out.print("Percurso BFS (Busca em Largura): ");
        arvore.mostrarBFS();
        break;

    case 7:
        System.out.print("Percurso DFS (Busca em Profundidade): ");
        arvore.mostrarDFS();
        break;

    case 8:
        arvore.mostrarAltura();
        break;

    case 9:
        arvore.mostrarEstrutura();
        break;

    case 10:
        System.out.println("Encerrando o programa ...");
        break;
```

```
        arvore.mostrarAltura();
        break;

    case 9:
        arvore.mostrarEstrutura();
        break;

    case 10:
        System.out.println("Encerrando o programa ... ");
        break;

    default:
        System.out.println("Opcao invalida! Tente novamente.");
        break;
    }
} while (opcao != 10);

scanner.close();
}

1 usage  victorhugoEngineer
private static void exibirMenu() {
    System.out.println("\n=====");
    System.out.println("    ARVORE BINARIA DE BUSCA (BST)");
    System.out.println("=====");
    System.out.println("1. Inserir valor");
    System.out.println("2. Buscar valor");
    System.out.println("3. Mostrar Pre-ordem (Raiz → Esq → Dir)");
    System.out.println("4. Mostrar Em ordem (Esq → Raiz → Dir)");
    System.out.println("5. Mostrar Pos-ordem (Esq → Dir → Raiz)");
    System.out.println("6. Mostrar BFS - Busca em Largura");
    System.out.println("7. Mostrar DFS - Busca em Profundidade");
    System.out.println("8. Mostrar altura da arvore");
    System.out.println("9. Mostrar estrutura da arvore");
    System.out.println("10. Sair");
    System.out.println("=====");
    System.out.print("Escolha uma opcao: ");
}
}
```


Parte II

Resultados

4 Resultado da Execução do Programa

Após a implementação das classes `No`, `ArvoreBinariaBusca` e `Main`, o programa pode ser executado por meio do método `main`.

Ao iniciar o programa, é apresentado ao usuário um menu interativo contendo todas as operações disponíveis para a Árvore Binária de Busca.

A Figura apresenta o resultado inicial da execução do programa no terminal.

4.0.1 Menu Principal

O menu apresentado ao usuário possui dez opções:

```
=====
ARVORE BINARIA DE BUSCA (BST)
=====
1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz -> Esq -> Dir)
4. Mostrar Em ordem (Esq -> Raiz -> Dir)
5. Mostrar Pos-ordem (Esq -> Dir -> Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair
-----
Escolha uma opcao:
```

O programa permanece aguardando uma entrada do usuário no momento em que apresenta a mensagem:

Escolha uma opcao:

A opção digitada determina qual operação será executada.

4.0.2 Inserção de Valores

Ao escolher a opção 1, o programa solicita ao usuário um valor:

Digite o valor a ser inserido:

O valor informado é encaminhado para o método:

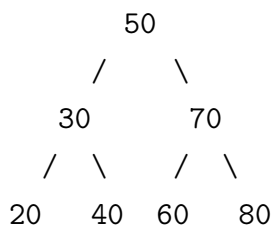
```
arvore.inserir(valor);
```

Esse método pertence à classe `ArvoreBinariaBusca` e realiza a inserção respeitando as propriedades da BST.

Por exemplo, considerando a inserção dos valores:

50, 30, 70, 20, 40, 60, 80

a árvore resultante será:



4.0.3 Busca de Valores

Ao selecionar a opção 2, o programa solicita um valor:

Digite o valor para buscar:

Em seguida, o método:

```
arvore.buscar(valor);
```

é utilizado para verificar se o elemento está presente na árvore.

Caso o valor seja encontrado, o programa apresenta uma mensagem indicando sucesso.

Caso contrário, informa que o elemento não foi encontrado.

A busca pode ser representada matematicamente por:

$$\text{Buscar}(x) = \begin{cases} \text{true}, & \text{se } x \in \text{árvore} \\ \text{false}, & \text{caso contrário} \end{cases}$$

4.0.4 Percursos da Árvore

As opções 3, 4 e 5 permitem visualizar a árvore utilizando os três principais percursos recursivos.

4.0.4.1 Pré-ordem

A opção 3 executa:

```
arvore.mostrarPreOrdem();
```

A ordem de visita é:

Raiz → Esquerda → Direita

Para a árvore apresentada anteriormente:

50, 30, 20, 40, 70, 60, 80

4.0.4.2 Em ordem

A opção 4 executa:

```
arvore.mostrarEmOrdem();
```

A ordem de visita é:

Esquerda → Raiz → Direita

O resultado será:

20, 30, 40, 50, 60, 70, 80

Uma característica importante da Árvore Binária de Busca é que o percurso em ordem apresenta os elementos em ordem crescente.

4.0.4.3 Pós-ordem

A opção 5 executa:

```
arvore.mostrarPosOrdem();
```

A ordem de visita é:

Esquerda → Direita → Raiz

Para a árvore utilizada como exemplo:

20, 40, 30, 60, 80, 70, 50

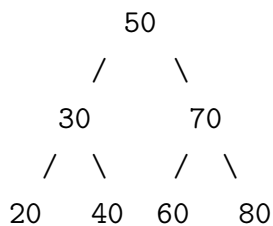
4.0.5 Busca em Largura – BFS

A opção 6 executa:

```
arvore.mostrarBFS();
```

O algoritmo BFS (*Breadth-First Search*) percorre a árvore nível por nível.

Para:



o resultado será:

50, 30, 70, 20, 40, 60, 80

A implementação utiliza uma fila (Queue), seguindo o princípio:

FIFO – First In, First Out

4.0.6 Busca em Profundidade – DFS

A opção 7 executa:

```
arvore.mostrarDFS();
```

O algoritmo DFS (*Depth-First Search*) percorre a árvore em profundidade utilizando uma pilha.

A implementação utiliza:

```
Stack<No> pilha = new Stack<>();
```

A pilha segue o princípio:

LIFO – Last In, First Out

Na implementação apresentada, o filho direito é colocado na pilha antes do filho esquerdo. Dessa forma, o filho esquerdo será processado primeiro.

O resultado produzido é equivalente ao percurso pré-ordem:

50, 30, 20, 40, 70, 60, 80

4.0.7 Cálculo da Altura

A opção 8 executa:

```
arvore.mostrarAltura();
```

Esse método calcula a altura da árvore utilizando recursão.

A convenção utilizada pelo programa é:

$$H(\text{árvore vazia}) = -1$$

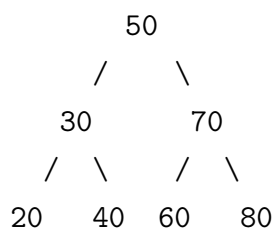
e:

$$H(\text{folha}) = 0$$

A fórmula utilizada é:

$$H(n) = 1 + \max(H(n_{\text{esquerda}}), H(n_{\text{direita}}))$$

Para a árvore:



a altura é:

$$H = 2$$

4.0.8 Visualização da Estrutura

A opção 9 executa:

```
arvore.mostrarEstrutura();
```

Esse método apresenta a organização hierárquica dos nós.

Para a árvore utilizada no exemplo, a saída será semelhante a:

```
--- Estrutura Hierarquica ---
(R) 50
    |--(E) 30
    |   |--(E) 20
    |   '--(D) 40
    '--(D) 70
        |--(E) 60
        '--(D) 80
-----
```

Os símbolos utilizados possuem os seguintes significados:

- (R) representa a raiz;
- (E) representa um filho esquerdo;
- (D) representa um filho direito.

4.0.9 Encerramento do Programa

A opção 10 encerra a execução do programa.

Quando o usuário seleciona essa opção, é apresentada a mensagem:

```
Encerrando o programa...
```

O laço principal utiliza a condição:

```
while (opcao != 10);
```

Portanto, enquanto:

$$\text{opcao} \neq 10$$

o menu continuará sendo exibido.

Quando:

$$\text{opcao} = 10$$

a condição se torna falsa e o programa encerra o laço.

4.1 Exemplo Completo de Execução

Considerando que o usuário insira os valores:

50, 30, 70, 20, 40, 60, 80

e posteriormente utilize as opções de visualização, os principais resultados serão:

Operação	Resultado
Inserção	50, 30, 70, 20, 40, 60, 80
Pré-ordem	50, 30, 20, 40, 70, 60, 80
Em ordem	20, 30, 40, 50, 60, 70, 80
Pós-ordem	20, 40, 30, 60, 80, 70, 50
BFS	50, 30, 70, 20, 40, 60, 80
DFS	50, 30, 20, 40, 70, 60, 80
Altura	2

4.2 Conclusão da Execução

O resultado apresentado no terminal demonstra que as três classes trabalham em conjunto.

A relação entre elas pode ser representada por:

Main → ArvoreBinariaBusca → No

A classe Main é responsável pela interação com o usuário, enquanto a classe ArvoreBinariaBusca realiza as operações sobre a árvore e a classe No representa cada elemento individual da estrutura.

Dessa forma, o programa implementa uma Árvore Binária de Busca com operações de inserção, busca, diferentes percursos, cálculo de altura e visualização da estrutura hierárquica.

```
=====
ARVORE BINARIA DE BUSCA (BST)
=====

1. Inserir valor
2. Buscar valor
3. Mostrar Pre-ordem (Raiz → Esq → Dir)
4. Mostrar Em ordem (Esq → Raiz → Dir)
5. Mostrar Pos-ordem (Esq → Dir → Raiz)
6. Mostrar BFS - Busca em Largura
7. Mostrar DFS - Busca em Profundidade
8. Mostrar altura da arvore
9. Mostrar estrutura da arvore
10. Sair

Escolha uma opcao: |
```

Figura 12 – Menu Do Terminal

4.3 Conclusão

O desenvolvimento deste projeto possibilitou a implementação e a compreensão dos principais conceitos relacionados às **Árvores Binárias de Busca (BST – Binary Search Tree)** em Java.

A estrutura foi desenvolvida utilizando três classes principais: `No`, `ArvoreBinariaBusca` e `Main`. Cada classe possui uma responsabilidade específica dentro do sistema, permitindo uma organização mais clara do código e uma melhor separação das funcionalidades.

A classe `No` representa a unidade básica da árvore. Cada objeto possui um valor e duas referências, uma para o filho esquerdo e outra para o filho direito. Dessa forma, os nós podem ser conectados para formar uma estrutura hierárquica:

$$No = (valor, esquerda, direita)$$

A classe `ArvoreBinariaBusca` é responsável pela implementação da lógica da árvore. Nela foram desenvolvidas operações de inserção e busca, além de diferentes formas de percorrer e analisar a estrutura. A inserção respeita a propriedade fundamental de uma BST:

$$\text{valores menores} \rightarrow \text{subárvore esquerda}$$

$$\text{valores maiores} \rightarrow \text{subárvore direita}$$

Essa propriedade também é utilizada durante a operação de busca, permitindo determinar em qual direção o algoritmo deve continuar sua procura.

Foram implementadas duas formas de busca: uma versão recursiva e uma versão iterativa. A versão recursiva utiliza chamadas sucessivas do método, enquanto a versão iterativa utiliza uma estrutura de repetição. Apesar das diferenças na implementação, ambas utilizam a propriedade da BST para localizar os elementos.

Também foram implementados diferentes percursos da árvore. O percurso **pré-ordem** segue a sequência:

Raiz $\rightarrow$ Esquerda $\rightarrow$ Direita

O percurso **em ordem** segue:

Esquerda $\rightarrow$ Raiz $\rightarrow$ Direita

e o percurso **pós-ordem** segue:

Esquerda $\rightarrow$ Direita $\rightarrow$ Raiz

Entre esses percursos, destaca-se o percurso em ordem, pois, em uma Árvore Binária de Busca, ele apresenta os elementos em ordem crescente.

Além dos percursos recursivos, foram implementadas as estratégias BFS e DFS. O BFS (*Breadth-First Search*) realiza a busca em largura, visitando os nós nível por nível e utilizando uma fila (Queue). Essa estrutura segue o princípio:

FIFO – First In, First Out

Já o DFS (*Depth-First Search*) realiza a busca em profundidade, utilizando uma pilha (Stack), que segue o princípio:

LIFO – Last In, First Out

Também foi implementado o cálculo da altura da árvore por meio de recursão. Considerando a convenção utilizada no projeto, uma árvore vazia possui altura -1 , enquanto uma árvore contendo apenas um nó possui altura 0 . A altura de um nó é determinada pela expressão:

$$H(n) = 1 + \max(H(n_{\text{esquerda}}), H(n_{\text{direita}}))$$

A classe Main, por sua vez, é responsável pela interação com o usuário. Por meio de um menu apresentado no terminal, o usuário pode inserir e buscar valores, executar os diferentes percursos, calcular a altura e visualizar a estrutura hierárquica da árvore.

A organização geral do projeto pode ser representada por:

```
Main → ArvoreBinariaBusca → No
```

Essa divisão permite separar a interface do usuário, a lógica da estrutura de dados e a representação individual dos elementos.

Considerando, por exemplo, a inserção dos valores:

50, 30, 70, 20, 40, 60, 80

a árvore formada é:

```

      50
     /  \
    30   70
   /  \  /  \
  20  40 60  80

```

A partir dessa estrutura, os principais resultados obtidos são:

Pré-ordem = 50, 30, 20, 40, 70, 60, 80

Em ordem = 20, 30, 40, 50, 60, 70, 80

Pós-ordem = 20, 40, 30, 60, 80, 70, 50

BFS = 50, 30, 70, 20, 40, 60, 80

DFS = 50, 30, 20, 40, 70, 60, 80

Altura = 2

Portanto, o projeto permitiu aplicar, de forma prática, conceitos fundamentais de **estruturas de dados, recursividade, pilhas, filas, árvores binárias e algoritmos de busca e percurso**.

Além de demonstrar o funcionamento de uma Árvore Binária de Busca, a implementação possibilitou compreender como diferentes algoritmos podem percorrer a mesma estrutura de maneiras distintas. A utilização de uma interface interativa também permitiu testar as operações diretamente e observar seus resultados.

Conclui-se, portanto, que a implementação realizada atende ao objetivo de demonstrar o funcionamento de uma Árvore Binária de Busca, desde a criação dos nós e organização dos elementos até a execução de operações de inserção, busca, percurso, cálculo da altura e visualização da estrutura.

No + BST + Algoritmos de percurso + Interface = Sistema completo de Árvore Binária de Busca
